

CODING TECHNICAL ASSESSMENT

Position: Full Stack Java Developer

Level: Mid-level

Technologies: Java, HTML, CSS, JavaScript

INSTRUCTIONS:

- This is a coding-focused technical assessment
- Complete all coding exercises and projects
- Submit your code files with clear documentation
- Time limit: 2-3 hours
- Reply to this email with your completed solutions

****Full Stack Java Developer Coding Assessment****

****Time Limit:** 3 hours**

****Instructions:****

- * Please complete all tasks and submit your code in the specified format.
- * Use Java 11 or later for coding exercises.
- * For practical projects, use your preferred Java-based web framework (e.g., Spring Boot, Play Framework).
- * Use a database of your choice (e.g., MySQL, PostgreSQL) for database-related tasks.
- * For front-end tasks, use HTML, CSS, and JavaScript.
- * Please follow the file naming conventions and submission guidelines.

****File Naming Conventions:****

- * For coding exercises, use the format `exercise\_<number>\_<your\_name>.java` (e.g., `exercise\_1\_john\_doe.java`).
- * For practical projects, use the format `project\_<number>\_<your\_name>.zip` (e.g., `project\_1\_john\_doe.zip`).
- * For code review tasks, use the format `review\_<number>\_<your\_name>.txt` (e.g., `review\_1\_john\_doe.txt`).

****Submission Guidelines:****

- * Please submit your code as a zip file.
- * Include a README file with instructions on how to run and test your code.
- * Make sure to include all dependencies and required libraries.

****CODING EXERCISES (7 questions)****

Exercise 1: Algorithm - Maximum Subarray Sum (15 minutes)

* Problem Statement: Given an array of integers, find the maximum contiguous subarray sum.

* Input/Output Examples:

`'2 -ç WC `[-2, -3, 4, -1, -2, 1, 5, -3]``

`'2 ÷WG WC `7` (subarray `[4, -1, -2, 1, 5]`)`

* Evaluation Criteria: Correctness, efficiency, and code quality.

* Sample Test Cases:

`'2 Pxercise_1_input1.txt` (array of 10 elements)`

`'2 Pxercise_1_input2.txt` (array of 100 elements)`

Exercise 2: Code Implementation - Singleton Pattern (20 minutes)

* Problem Statement: Implement the Singleton pattern in Java to ensure a class has only one instance.

* Input/Output Examples:

`'2 -ç WC `Singleton.getInstance()``

`'2 ÷WG WC `Singleton instance``

* Evaluation Criteria: Correctness, thread-safety, and code quality.

* Sample Test Cases:

`'2 Pxercise_2_input1.txt` (multiple threads accessing the Singleton instance)`

`'2 Pxercise_2_input2.txt` (concurrent access to the Singleton instance)`

Exercise 3: Bug Fixing - NullPointerException (20 minutes)

* Problem Statement: Fix the bug in the given Java code that causes a `NullPointerException`.

* Input/Output Examples:

'2 -ç WC `User user = new User(); user.getName();`

'2 ÷WG WC `John Doe` (instead of `NullPointerException`)

* Evaluation Criteria: Correctness, debugging skills, and code quality.

* Sample Test Cases:

'2 Pexercice_3_input1.txt` (initial code with bug)

'2 Pexercice_3_input2.txt` (fixed code)

Exercise 4: Code Optimization - Database Query (25 minutes)

* Problem Statement: Optimize the given database query to reduce execution time.

* Input/Output Examples:

'2 -ç WC `SELECT \* FROM users WHERE age > 18`

'2 ÷WG WC `Optimized query with reduced execution time`

* Evaluation Criteria: Correctness, performance improvement, and code quality.

* Sample Test Cases:

'2 Pexercice_4_input1.txt` (initial query with slow execution time)

'2 Pexercice_4_input2.txt` (optimized query with fast execution time)

Exercise 5: API Development - RESTful API (30 minutes)

* Problem Statement: Develop a RESTful API using Java to manage users.

* Input/Output Examples:

'2 -ç WC `POST /users { "name": "John Doe", "age": 30 }`

'2 ÷WG WC `User created successfully`

* Evaluation Criteria: Correctness, API design, and code quality.

* Sample Test Cases:

'2 Pexercice_5_input1.txt` (create user)

'2 Pexercice_5_input2.txt` (get user by ID)

Exercise 6: Database Query Challenge (25 minutes)

* Problem Statement: Write a SQL query to retrieve the top 3 products with the highest sales.

* Input/Output Examples:

'2 -ç WC `SELECT \* FROM products WHERE sales > 1000`

'2 ÷WG WC `Top 3 products with the highest sales`

* Evaluation Criteria: Correctness, query efficiency, and code quality.

* Sample Test Cases:

'2 Pxercise_6_input1.txt` (initial query with slow execution time)

'2 Pxercise_6_input2.txt` (optimized query with fast execution time)

Exercise 7: Code Implementation - Factory Pattern (20 minutes)

* Problem Statement: Implement the Factory pattern in Java to create objects without specifying the exact class of object.

* Input/Output Examples:

'2 -ç WC `Vehicle vehicle = VehicleFactory.createVehicle("car");`

'2 ÷WG WC `Car`

* Evaluation Criteria: Correctness, code quality, and design principles.

* Sample Test Cases:

'2 Pxercise_7_input1.txt` (create car object)

'2 Pxercise_7_input2.txt` (create truck object)

PRACTICAL PROJECTS (3 mini-projects)

Project 1: To-Do List App (45 minutes)

* Problem Statement: Develop a simple To-Do List app using Java and HTML/CSS/JavaScript.

* Requirements:

'2 7&V FR W6W" -çFW&`ace to add, remove, and mark tasks as completed.

'2 7F÷&R F 6·2 -â F F & 6P.

'2 -x ÆEVÖVçB W6W" WF†VçF-6 F-öââ

* Evaluation Criteria: Correctness, functionality, and code quality.

* Sample Test Cases:

'2 &ö!V7Eó ö-ç WC çG‡F † FB F 6²•

'2 &ö!V7Eó ö-ç WC"çG‡F ‡&VÖðve task)

Project 2: Weather API (45 minutes)

* Problem Statement: Develop a RESTful API using Java to retrieve weather data from an external API.

* Requirements:

'2 -çFVprate with an external weather API (e.g., OpenWeatherMap).

'2 -x ÆVÖVçB ' VæG ö-çG2 Fò &WG ieve current weather and forecast data.

'2 † æFÆER W'&÷'2 æB Pxceptions.

* Evaluation Criteria: Correctness, API design, and code quality.

* Sample Test Cases:

'2 &ö!V7Eó%ö-ç WC çG‡F †vWB 7W'&VçB peather)

'2 &ö!V7Eó%ö-ç WC"çG‡F †vWB `orecast data)

Project 3: Chat Application (60 minutes)

* Problem Statement: Develop a simple chat application using Java and HTML/CSS/JavaScript.

* Requirements:

'2 7&V FR W6W" -çFW&`ace to send and receive messages.

'2 7F÷&R ÖW76 vW2 -â F F & 6P.

'2 -x ÆVÖVçB &V ÂxF-ÖR ÖW76 v-æR W6-æR WebSockets.

* Evaluation Criteria: Correctness, functionality, and code quality.

* Sample Test Cases:

'2 &ö!V7Eó5ö-ç WC çG‡F ‡6VæB ÖW76 vR•

'2 &ö!V7Eó5ö-ç WC"çG‡F ‡&V6V—`e message)

CODE REVIEW TASKS (3 questions)

Review 1: Code Quality Review (20 minutes)

* Problem Statement: Review the given Java code and suggest improvements for code quality, readability, and maintainability.

* Input/Output Examples:

'2 -ç WC `public class User { private String name; ... }`

'2 ÷WG WC `Improved code with better naming conventions, comments, and formatting`

* Evaluation Criteria: Correctness, code quality, and suggestions for improvement.

* Sample Test Cases:

'2 &Pview_1_input1.txt` (initial code with poor quality)

'2 &Pview_1_input2.txt` (improved code with good quality)

Review 2: Performance Optimization Review (25 minutes)

* Problem Statement: Review the given Java code and suggest improvements for performance optimization.

* Input/Output Examples:

'2 -ç WC `public class User { public void setName(String name) { ... } }`

'2 ÷WG WC `Improved code with better performance`

* Evaluation Criteria: Correctness, performance improvement, and suggestions for optimization.

* Sample Test Cases:

'2 &Pview_2_input1.txt` (initial code with poor performance)

'2 &Pview_2_input2.txt` (improved code with good performance)

Review 3: Design Principles Review (25 minutes)

* Problem Statement: Review the given Java code and suggest improvements for design principles, such as SOLID principles.

* Input/Output Examples:

'2 -ç WC `public class User { public void save() { ... } }`

'2 ÷WG WC `Improved code with better design principles`

* Evaluation Criteria: Correctness, design principles, and suggestions for improvement.

* Sample Test Cases:

'2 &Pview_3_input1.txt` (initial code with poor design principles)

'2 &Pview_3_input2.txt` (improved code with good design principles)

Generated on: 15/09/2025 14:34:39